Software Re-engineering

Fall 2026

Instructor Introduction

- **Dr. Muhammad Fawad**
  - Assistant Professor
- **Post-Doc** – Software Engineering
- **PhD** – Software Engineering
- **PGD** – Professional Ethics and Teaching Methodology
- **MS** – Software Engineering
- **BS** – Software Engineering
- E-mail: muhammadfawad@cuilahore.edu.pk

Course Objectives & Course Outline

Course Introduction

Course	Software Re-Engineering
Course Type	SE - Core
Credit Hours	3
Lecture Duration	3 hours per week
Lab Duration	-
Semester Duration	16 weeks

Course Description

- This course provides the concepts and application of software re-engineering techniques.
- Topics include: Introduction to Software Re-Engineering; Software Evolution & Legacy Systems; Re-Engineering Techniques; Refactoring & Applications; Re-Architecting; Big Rewrite; Project Improvement & Automating the Development Environment; Object Oriented Re-Engineering Patterns; Quality Issues; Tool Support; Challenges & Stakeholder Aspiration; Software Maintenance; and Re-Engineering Economics.

Unit wise Major Topics

Unit	Topic	No. of Teaching Hours
1.	Software Re-Engineering: Introduction, Evolution, and Legacy Systems.	3
2.	Techniques of Re-Engineering: Reverse Engineering, Restructuring, and Forward Propagation.	7.5
3.	Refactoring & Applications; Re-Architecting; and Big Rewrite.	6
4.	Project Improvement & Automating the Development Environment.	6
5.	Object Oriented Re-engineering Patterns.	7.5
6.	Quality Issues in Re-Engineering Processes.	4.5
7.	Tool Support for Re-engineering; and Challenges & Stakeholder Aspiration.	6
8.	Software Maintenance; and Re-engineering Economics.	4.5
Total Contact Hours		**45**

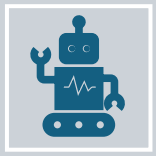
Mapping of CLOs and SOs

Mapping of CLOs and SOs				
Sr.#	**Unit #**	**Course Learning Outcomes**	**Blooms Taxonomy Learning Level**	**SO**
CLO-1	1	Recognize the concepts of software re-engineering.	*Understanding*	1
CLO-2	2-4	Apply re-engineering techniques to maintain and modify software systems.	*Applying*	2,4
CLO-3	5	Apply Object oriented re-engineering patterns to maintenance problems.	*Applying*	2,4
CLO-4	6	Illustrate quality and tools support issues and challenges.	*Applying*	2,4
CLO-5	3-8	Reconstruct low-level source code structure of a legacy system by applying refactoring strategies and	*Creating*	2,4,9

CLO Assessment Mechanism

CLO Assessment Mechanism					
Assessment Tools	**CLO-1**	**CLO-2**	**CLO-3**	**CLO-4**	**CLO-5**
Quizzes	Quiz 1	Quiz 2	Quiz 3	Quiz 4	-
Assignments	-	Assignment 1	Assignment 2	Assignment 3	Assignment 4
Mid Term Exam	Mid Term Exam	Mid Term Exam	Mid Term Exam	-	-
Final Term Exam	Final Term Exam				

Course Books



"Software Reengineering" by Robert S. Arnold



"Refactoring: Improving the Design of Existing Code" by Martin Fowler



"Legacy Code: A Programmer's Guide to Reengineering" by Michael Feathers

Course Policies

- **Late Work:**
  - No late submissions will be accepted under any circumstances.
- **Attendance Requirement:**
  - Students must maintain a minimum of 80% attendance to be eligible to appear in the final exam.
- **Makeup Policy:**
  - No makeup quizzes or assignments will be conducted.

Google Classroom

You are expected to regularly check GCR for:

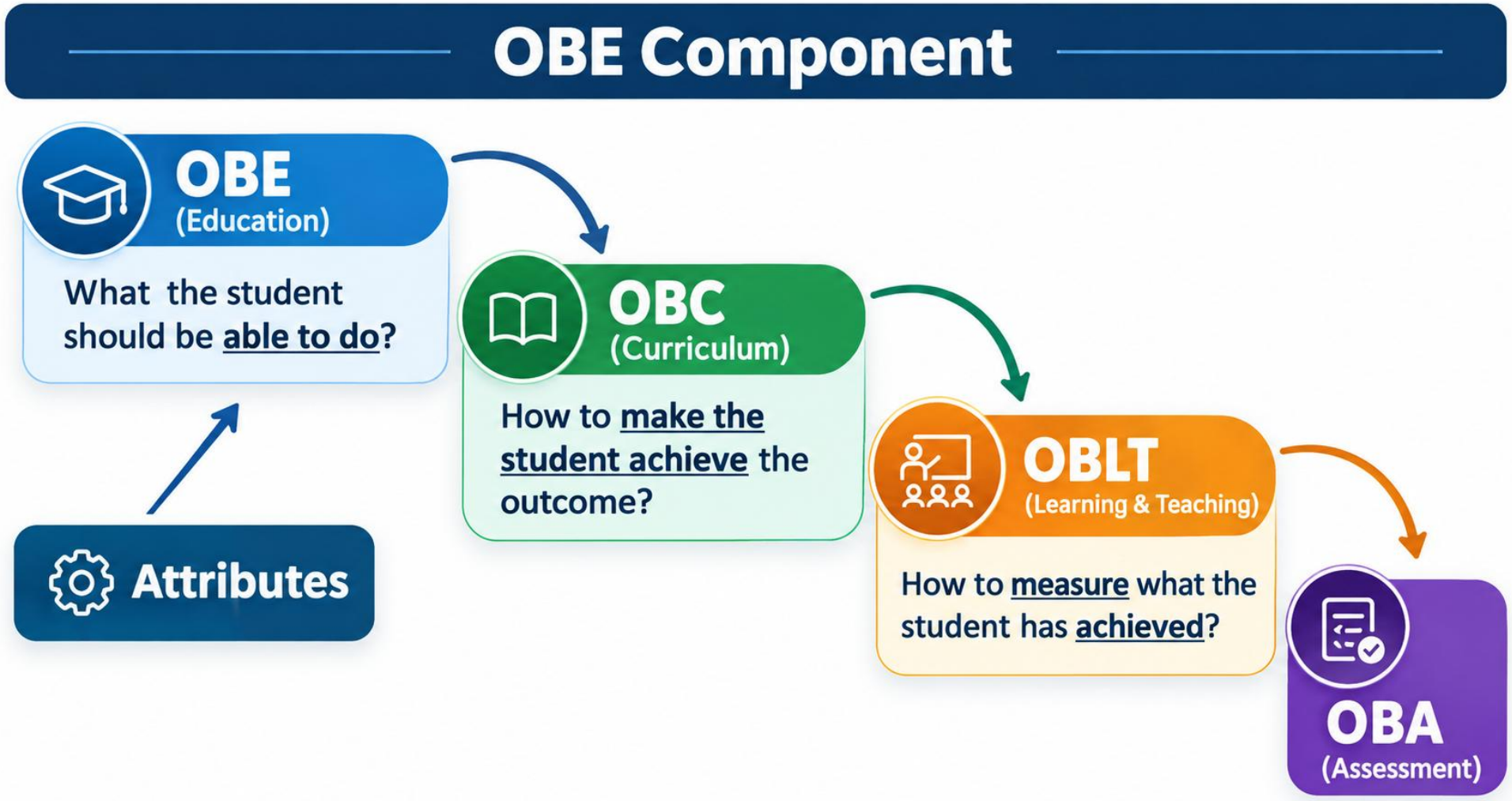
- Supporting Material
- Resources
- Announcements

https://classroom.google.com/

Course Objectives

1. Understand the need for software re-engineering and the contexts in which it is applied.
2. Apply reverse engineering techniques to understand the structure and behavior of existing software systems.
3. Restructure and refactor software systems to improve quality attributes of a software system.
4. Manage the process of migrating legacy systems to modern platforms.
5. Evaluate and use various re-engineering tools and techniques effectively.

OBE Based Learning



BLOOM'S TAXONOMY (REVISED)

A hierarchy of cognitive skills from lower-order to higher-order thinking.

LOWER-ORDER THINKING SKILLS ⟶ HIGHER-ORDER THINKIEING SKILLS

COGNITIVE LEVEL	1 REMEMBER	2 UNDERSTAND	3 APPLY	4 ANALYZE	5 EVALUATE	6 CREATE
FOCUS	Recall facts and basic concepts	Understand meaning and explain ideas	Use information in new situations	Break information into parts and explore relationships	Make judgements based on criteria and standards	Create new ideas, products or solutions
EXAMPLES	• Define • Identify • List • Label • Match • Name • Recall • Recognize • Repeat • State	• Classify • Compare • Describe • Discuss • Explain • Illustrate • Paraphrase • Summarize • Translate	• Apply • Calculate • Compute • Develop • Execute • Implement • Operate • Practice • Solve • Use	• Analyze • Compare • Contrast • Differentiate • Examine • Identify • Infer • Organize • Question • Investigate	• Assess • Critique • Evaluate • Judge • Justify • Measure • Recommend • Support • Test	• Design • Generate • Plan • Produce • Develop • Construct • Formulate • Invent • Compose

USED IN

- Assessments & Exams
- Lesson Planning & Instruction
- Curriculum Design & Learning Outcomes
- Critical Thinking & Problem Solving
- Performance Evaluation
- Innovation & Creativity

Adapted from Anderson & Krathwohl (2001): A Taxonomy for Learning, Teaching, and Assessing.

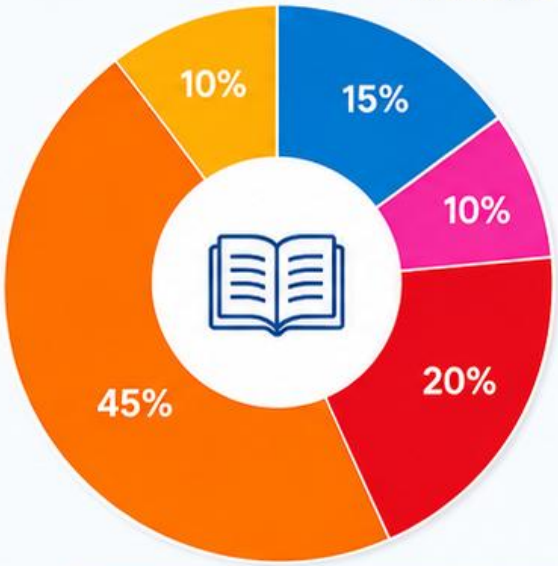
PERCENTAGE OF BLOOM'S LEVELS BY COURSE CATEGORY

The charts below illustrate the distribution of Bloom's Taxonomy levels across different course categories.

DOMAIN CORE COURSES

PERCENTAGE OF BLOOM'S LEVELS:

DOMAIN CORE



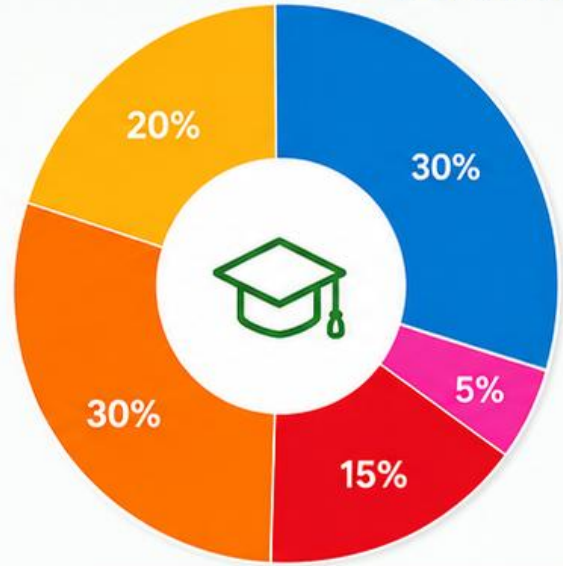
BLOOM'S LEVEL	PERCENTAGE
Creating	15%
Evaluating	10%
Analyzing	20%
Applying	45%
Understanding	10%

Core courses emphasize **Applying** and **Analyzing** skills, supporting practical and analytical competencies.

DOMAIN ELECTIVE COURSES

PERCENTAGE OF BLOOM'S LEVELS:

DOMAIN ELECTIVE



BLOOM'S LEVEL	PERCENTAGE
Creating	30%
Evaluating	5%
Analyzing	15%
Applying	30%
Understanding	20%

Elective courses emphasize **Creating** and **Applying** skills, fostering innovation and advanced application.

Creating
Generate new ideas, design, innovate

Evaluating
Make judgments, assess, justify

Analyzing
Examine, compare, identify patterns

Applying
Use knowledge in new situations

Understanding
Explain, summarize, interpret

DIFFERENCE BETWEEN COURSE OBJECTIVES AND COURSE LEARNING OUTCOMES

ASPECT	COURSE OBJECTIVES	COURSE LEARNING OUTCOMES
DEFINITION	Broad goals of what the course aims to achieve.	Specific skills and knowledge students should acquire.
FOCUS	Overall intentions of the course.	Measurable student achievements.
NATURE	General and often qualitative.	Specific and often quantitative.
SCOPE	Broad and overarching.	Narrow and detailed.
MEASUREMENT	Difficult to measure directly.	Easier to measure through assessments.
EXAMPLES	• Understand the principles of computer science. • Develop problem-solving skills.	• Explain the basic concepts of algorithms. • Implement basic sorting algorithms in a programming language.
AUDIENCE	Instructors and course designers.	Students and instructors.
TIME FRAME	Long-term goals.	Achievable within the duration of the course.

Any Questions?